

ASSIGNMENT 12.5

Roll Number: 2303A51210

27-02-26

Batch – 04

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- Expected Output:
 - o A functional Python script implementing Merge Sort with proper documentation

prompt:

#generate a python code to perform merge sort with function name `merge_sort(arr)` that sorts the input array in ascending order and include time and space complexity in function docstring.

inputs:

`arr = [38, 27, 43, 3, 9, 82, 10]`

Output:

```
● PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & 215.py
Sorted array: [3, 9, 10, 27, 38, 43, 82]
○ PS C:\Users\Vishn\OneDrive\Desktop\AIAC>
```

Code:

```
1 #generate a python code to perform merge sort with function name merge_sort(arr) th
2 def merge_sort(arr):
3     """
4     Sorts the input array in ascending order using the merge sort algorithm.
5
6     Time Complexity: O(n log n) - where n is the number of elements in the array.
7     Space Complexity: O(n) - due to the temporary arrays used for merging.
8
9     Parameters:
10    arr (list): The list of elements to be sorted.
11
12    Returns:
13    list: A new list containing the sorted elements.
14    """
15    if len(arr) <= 1:
16        return arr
17
18    mid = len(arr) // 2
19    left_half = merge_sort(arr[:mid])
20    right_half = merge_sort(arr[mid:])
21
22    return merge(left_half, right_half)
23
24 def merge(left, right):
25     """Merges two sorted lists into a single sorted list.
26     """
27     merged = []
28     left_index = right_index = 0
29
30     while left_index < len(left) and right_index < len(right):
31         if left[left_index] < right[right_index]:
32             merged.append(left[left_index])
33             left_index += 1
34         else:
35             merged.append(right[right_index])
36             right_index += 1
37
38     # If there are remaining elements in the left half, add them to merged
39     while left_index < len(left):
40         merged.append(left[left_index])
41         left_index += 1
42
43     # If there are remaining elements in the right half, add them to merged
44     while right_index < len(right):
45         merged.append(right[right_index])
46         right_index += 1
47
48     return merged
49
50 # Example usage:
51 if __name__ == "__main__":
52     arr = [38, 27, 43, 3, 9, 82, 10]
53     sorted_arr = merge_sort(arr)
54     print("Sorted array:", sorted_arr)
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.
 - o Test with various inputs.
- Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

prompt:

#generate a python code that performs binary search with function `binary_search(arr, target)` that returns the index of target in arr if found, otherwise returns -1. and include docstrings explaining best, average, and worst-case complexities . take target as dynamic input from the user.

Code:

```
def binary_search(arr, target):
    """
    Perform binary search on a sorted array to find the index of the target element.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for in the array.

    Returns:
    int: The index of the target element if found, otherwise -1.

    Complexity:
    - Best Case: O(1) when the target is at the middle of the array.
    - Average Case: O(log n) where n is the number of elements in the array.
    - Worst Case: O(log n) when the target is not present in the array or is at one of the ends.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1

# Example usage
if __name__ == "__main__":
    # Sample sorted array
    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # Take target input from the user
    target = int(input("Enter the target element to search for: "))

    # Perform binary search and print the result
    result = binary_search(arr, target)
    if result != -1:
        print(f"Element {target} found at index: {result}")
    else:
        print(f"Element {target} not found in the array.")
```

Output:

```
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/Users/Vishn/OneDrive/Desktop/AIAC/2303a51215.py
Enter the target element to search for: -3
Element -3 not found in the array.
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/Users/Vishn/OneDrive/Desktop/AIAC/2303a51215.py
Enter the target element to search for: 3
Element 3 found at index: 2
PS C:\Users\Vishn\OneDrive\Desktop\AIAC>
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
 2. Sort appointments based on time or consultation fee. Student Task
- Use AI to recommend suitable searching and sorting algorithms.
 - Justify the selected algorithms.
 - Implement the algorithms in Python

prompt:

#generate a python code for Smart Healthcare Appointment Scheduling System containing ppointment ID, patient name, doctor name, appointment time, and consultation fee . search for appointments by appointment ID and sort appointments based on appointment time and fee

Code:

```

2 class Appointment:
3     def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
4         self.appointment_id = appointment_id
5         self.patient_name = patient_name
6         self.doctor_name = doctor_name
7         self.appointment_time = appointment_time
8         self.consultation_fee = consultation_fee
9
10 class AppointmentScheduler:
11     def __init__(self):
12         self.appointments = []
13     def add_appointment(self, appointment):
14         self.appointments.append(appointment)
15     def search_appointment_by_id(self, appointment_id):
16         for appointment in self.appointments:
17             if appointment.appointment_id == appointment_id:
18                 return appointment
19         return None
20     def sort_appointments_by_time(self):
21         self.appointments.sort(key=lambda x: x.appointment_time)
22     def sort_appointments_by_fee(self):
23         self.appointments.sort(key=lambda x: x.consultation_fee)
24
25 # Example usage
26 scheduler = AppointmentScheduler()
27 scheduler.add_appointment(Appointment(1, "John Doe", "Dr. Smith", "2023-10-01 10:00", 100))
28 scheduler.add_appointment(Appointment(2, "Jane Doe", "Dr. Brown", "2023-10-01 11:00", 150))
29 scheduler.add_appointment(Appointment(3, "Alice", "Dr. Green", "2023-10-01 09:00", 120))
30 # Search for an appointment by ID
31 appointment = scheduler.search_appointment_by_id(2)
32 if appointment:
33     print(f"Appointment found: {appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for ${appointment.consultation_fee}")
34 else:
35     print("Appointment not found.")
36 # Sort appointments by time
37 scheduler.sort_appointments_by_time()
38 print("Appointments sorted by time:")
39 for appointment in scheduler.appointments:
40     print(f"{appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for ${appointment.consultation_fee}")
41 # Sort appointments by fee
42 scheduler.sort_appointments_by_fee()
43 print("Appointments sorted by fee:")
44
45 for appointment in scheduler.appointments:
46     print(f"{appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for ${appointment.consultation_fee}")
47

```

Output:

```

PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/User
Appointment found: Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
Appointments sorted by time:
Alice with Dr. Green at 2023-10-01 09:00 for $120
John Doe with Dr. Smith at 2023-10-01 10:00 for $100
Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
Appointments sorted by fee:
John Doe with Dr. Smith at 2023-10-01 10:00 for $100
Alice with Dr. Green at 2023-10-01 09:00 for $120
Jane Doe with Dr. Brown at 2023-10-01 11:00 for $150
PS C:\Users\Vishn\OneDrive\Desktop\AIAC>

```

Task Description #4: Railway Ticket Reservation System Scenario

A railway reservation system stores booking details such as ticket

ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.

- Justify the algorithm choices.
- Implement searching and sorting in Python

Code:

```
class Reservation:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date

class RailwayReservationSystem:
    def __init__(self):
        self.reservations = []

    def add_reservation(self, reservation):
        self.reservations.append(reservation)

    def search_by_ticket_id(self, ticket_id):
        for reservation in self.reservations:
            if reservation.ticket_id == ticket_id:
                return reservation
        return None

    def sort_by_travel_date(self):
        self.reservations.sort(key=lambda x: x.travel_date)

    def sort_by_seat_number(self):
        self.reservations.sort(key=lambda x: x.seat_number)

# Example usage
if __name__ == "__main__":
    system = RailwayReservationSystem()

    reservation1 = Reservation("T001", "Alice", "Train100", "A1", "2024-07-01")
    reservation2 = Reservation("T002", "Bob", "Train101", "B2", "2024-07-02")
    reservation3 = Reservation("T003", "Charlie", "Train102", "C3", "2024-07-01")

    system.add_reservation(reservation1)
    system.add_reservation(reservation2)
    system.add_reservation(reservation3)

    # Search by ticket ID
    search_result = system.search_by_ticket_id("T002")
    if search_result:
        print(f"Found reservation: {search_result.passenger_name} on {search_result.train_number}")
    else:
        print("Reservation not found.")

    # Sort by travel date
    system.sort_by_travel_date()
    print("Reservations sorted by travel date:")
    for res in system.reservations:
        print(f"{res.passenger_name} - {res.travel_date}")

    # Sort by seat number
    system.sort_by_seat_number()
    print("Reservations sorted by seat number:")
    for res in system.reservations:
        print(f"{res.passenger_name} - {res.seat_number}")
```

Output:

```
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/Users/Vishn/OneDrive/Desktop/AIAC/2303a51215.py
Found reservation: Bob on Train101
Reservations sorted by travel date:
Alice - 2024-07-01
Charlie - 2024-07-01
Bob - 2024-07-02
Reservations sorted by seat number:
Alice - A1
Bob - B2
Charlie - C3
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> █
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

Prompt:

#generate a python code for Smart Hostel Room Allocation System which search allocation details based on student ID and sorts based on room number and allocation date give me a optimized code

Code:


```

class HostelRoomAllocation:
    def __init__(self):
        self.allocations = []

    def add_allocation(self, student_id, room_number, allocation_date):
        allocation = {
            'student_id': student_id,
            'room_number': room_number,
            'allocation_date': allocation_date
        }
        self.allocations.append(allocation)

    def search_allocation_by_student_id(self, student_id):
        return [allocation for allocation in self.allocations if allocation['student_id'] == student_id]

    def sort_allocations(self):
        return sorted(self.allocations, key=lambda x: (x['room_number'], x['allocation_date']))

# Example usage
if __name__ == "__main__":
    hostel = HostelRoomAllocation()
    hostel.add_allocation('S001', '101', '2023-01-01')
    hostel.add_allocation('S002', '102', '2023-01-02')
    hostel.add_allocation('S001', '103', '2023-01-03')

    print("Allocations for student S001:")
    print(hostel.search_allocation_by_student_id('S001'))

    print("\nSorted Allocations:")
    print(hostel.sort_allocations())

```

Output:

```

PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/Users/Vishn/OneDrive/Desktop/AIAC/2303a51215.py
Allocations for student S001:
[{'student_id': 'S001', 'room_number': '101', 'allocation_date': '2023-01-01'}, {'student_id': 'S001', 'room_number': '103', 'allocation_date': '2023-01-03'}]

Sorted Allocations:
[{'student_id': 'S001', 'room_number': '101', 'allocation_date': '2023-01-01'}, {'student_id': 'S002', 'room_number': '102', 'allocation_date': '2023-01-02'}, {'student_id': 'S001', 'room_number': '103', 'allocation_date': '2023-01-03'}]
PS C:\Users\Vishn\OneDrive\Desktop\AIAC>

```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

Prompt:

#generate a python code for Online Movie Streaming Platform which contains with movie ID, title, genre, rating, and release year and search movie based on movie name and sorts based on rating give me a optimized code

Code:

```
class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year

class MovieStreamingPlatform:
    def __init__(self):
        self.movies = []
    def add_movie(self, movie):
        self.movies.append(movie)
    def search_movie_by_title(self, title):
        return [movie for movie in self.movies if title.lower() in movie.title.lower()]
    def sort_movies_by_rating(self):
        return sorted(self.movies, key=lambda x: x.rating, reverse=True)

# Example usage
if __name__ == "__main__":
    platform = MovieStreamingPlatform()
    platform.add_movie(Movie(1, "Inception", "Sci-Fi", 8.8, 2010))
    platform.add_movie(Movie(2, "The Dark Knight", "Action", 9.0, 2008))
    platform.add_movie(Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014))

    search_results = platform.search_movie_by_title("Inception")
    print("Search Results:")
    for movie in search_results:
        print(f"{movie.title} ({movie.release_year}) - Rating: {movie.rating}")

    sorted_movies = platform.sort_movies_by_rating()
    print("\nMovies sorted by rating:")
    for movie in sorted_movies:
        print(f"{movie.title} ({movie.release_year}) - Rating: {movie.rating}")
```

Output:

```
Search Results:
Inception (2010) - Rating: 8.8

Movies sorted by rating:
The Dark Knight (2008) - Rating: 9.0
Inception (2010) - Rating: 8.8
Interstellar (2014) - Rating: 8.6
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> |
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Prompt:

#generate a python code for Smart Agriculture Crop Monitoring System which contains crop ID, crop name, soil moisture level, temperature, and yield estimate and search crop details using crop ID. and sort crops based on moisture level or yield estimate.

Code:

```
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture = soil_moisture
        self.temperature = temperature
        self.yield_estimate = yield_estimate

class CropMonitoringSystem:
    def __init__(self):
        self.crops = []
    def add_crop(self, crop):
        self.crops.append(crop)
    def search_crop_by_id(self, crop_id):
        for crop in self.crops:
            if crop.crop_id == crop_id:
                return crop
        return None
    def sort_crops_by_moisture(self):
        return sorted(self.crops, key=lambda x: x.soil_moisture)
    def sort_crops_by_yield_estimate(self):
        return sorted(self.crops, key=lambda x: x.yield_estimate)

# Example usage
if __name__ == "__main__":
    system = CropMonitoringSystem()
    system.add_crop(Crop(1, "Wheat", 30, 25, 100))
    system.add_crop(Crop(2, "Corn", 40, 28, 150))
    system.add_crop(Crop(3, "Rice", 20, 22, 120))

    crop = system.search_crop_by_id(2)
    if crop:
        print(f"Crop found: {crop.crop_name}, Soil Moisture: {crop.soil_moisture}, Temperature: {crop.temperature}, Yield Estimate: {crop.yield_estimate}")
    else:
        print("Crop not found.")

    print("\nCrops sorted by soil moisture:")
    for crop in system.sort_crops_by_moisture():
        print(f"{crop.crop_name}: {crop.soil_moisture}")

    print("\nCrops sorted by yield estimate:")
    for crop in system.sort_crops_by_yield_estimate():
        print(f"{crop.crop_name}: {crop.yield_estimate}")
```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Prompt:

#generate a python code for Airport Flight Management System which contains flight ID, airline name, departure time, arrival time, and status. and search flight details using flight ID.and Sort flights based on departure time or arrival time.

Code:

```
class Flight:
    def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status

class AirportFlightManagementSystem:
    def __init__(self):
        self.flights = []

    def add_flight(self, flight):
        self.flights.append(flight)

    def search_flight_by_id(self, flight_id):
        for flight in self.flights:
            if flight.flight_id == flight_id:
                return flight
        return None

    def sort_flights_by_departure_time(self):
        self.flights.sort(key=lambda x: x.departure_time)

    def sort_flights_by_arrival_time(self):
        self.flights.sort(key=lambda x: x.arrival_time)

# Example usage
if __name__ == "__main__":
    system = AirportFlightManagementSystem()
    system.add_flight(Flight("FL001", "Airline A", "10:00", "12:00", "On Time"))
    system.add_flight(Flight("FL002", "Airline B", "11:00", "13:00", "Delayed"))
    system.add_flight(Flight("FL003", "Airline C", "09:00", "11:00", "On Time"))

    flight = system.search_flight_by_id("FL002")
    if flight:
        print(f"Flight ID: {flight.flight_id}, Airline: {flight.airline_name}, Departure: {flight.departure_time}, Arrival: {flight.arrival_time}, Status: {flight.status}")
    else:
        print("Flight not found.")

    print("\nFlights sorted by departure time:")
    system.sort_flights_by_departure_time()
    for flight in system.flights:
        print(f"Flight ID: {flight.flight_id}, Departure Time: {flight.departure_time}")

    print("\nFlights sorted by arrival time:")
    system.sort_flights_by_arrival_time()
    for flight in system.flights:
        print(f"Flight ID: {flight.flight_id}, Arrival Time: {flight.arrival_time}")
```

Output:

```
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/Users/Vishn/OneDrive/Desktop/AIAC/2303a51215.py  
Flight ID: FL002, Airline: Airline B, Departure: 11:00, Arrival: 13:00, Status: Delayed
```

```
Flights sorted by departure time:  
Flight ID: FL003, Departure Time: 09:00  
Flight ID: FL001, Departure Time: 10:00  
Flight ID: FL002, Departure Time: 11:00
```

```
Flights sorted by arrival time:  
Flight ID: FL003, Arrival Time: 11:00  
Flight ID: FL001, Arrival Time: 12:00  
Flight ID: FL002, Arrival Time: 13:00  
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> █
```

```
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> & C:/Python314/python.exe c:/Users/Vishn/OneDrive/Desktop/AIAC/2303a51215.py  
Flight ID: FL002, Arrival Time: 13:00  
PS C:\Users\Vishn\OneDrive\Desktop\AIAC> █
```